Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

1. In 0 based indexing , mapping the root i to its left child is

a) trunc(i/2)

b) 2*i

c) 2*i+1

d) Round(i/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

1. In 0 based indexing , mapping the root i to its left child is

a) trunc(i/2)

b) 2*i

c) 2*i+1

d) Round(i/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

2. In node based array representation of a tree, a leaf is identified when its

a) left = -1 and right = -1

b) left = 0 and right = 0

c) Left = -1 is sufficient

d) Left = 0 is sufficient

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

2. In node based array representation of a tree, a leaf is identified when its

a) left = -1 and right = -1

b) left = 0 and right = 0

c) Left = -1 is sufficient

d) Left = 0 is sufficient

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

3. In 1 based indexing , the highest index in the array representation of a complete binary tree of n nodes is

a) n

b) n - 1

c) Log(n, 2)

d) n + 1

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

3. In 1 based indexing , the highest index in the array representation of a complete binary tree of n nodes is

a) n

b) n - 1

c) Log(n, 2)

d) n + 1

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

4. In 1 based indexing , the leaf node i in the array representation of a complete binary tree of n nodes will satisfy the following property

a) i > n

b) 2i > n

c) 2i >= n

d) i > 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

4. In 1 based indexing , the leaf node i in the array representation of a complete binary tree of n nodes will satisfy the following property

a) i > n

b) 2i > n

c) 2i >= n

d) i > 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

5. In 1 based indexing , a tree of n nodes which is not a complete tree may have a node stored at the highest index

a) n

b) 2^n -1

c) 2*n

d) 2^n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

5. In 1 based indexing , a tree of n nodes which is not a complete tree may have a node stored at the highest index

a) n

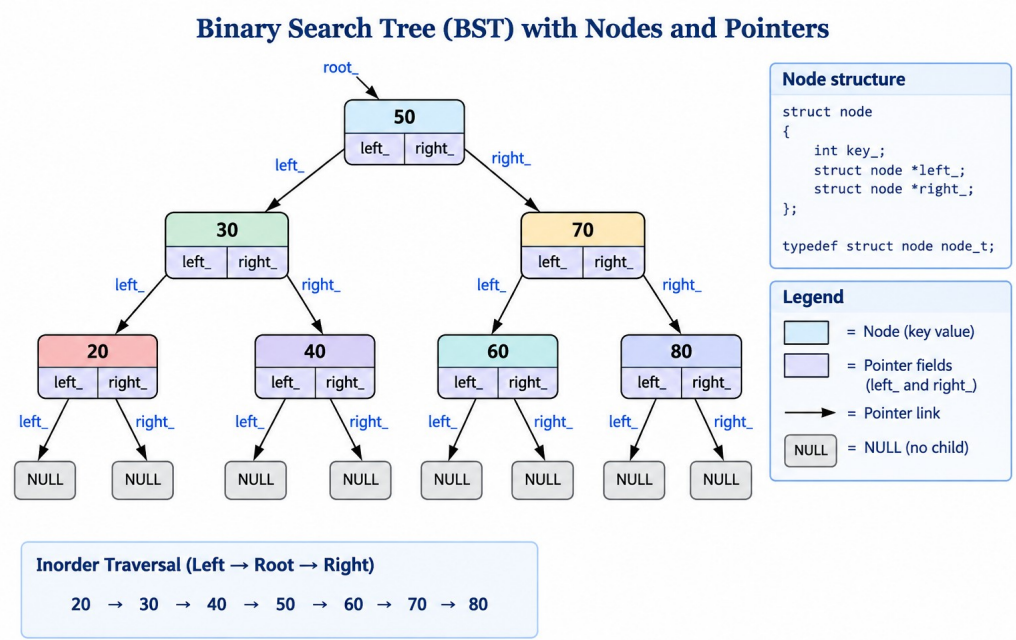
b) 2^n -1

c) 2*n

d) 2^n

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation



© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

1. Based on relative position of parent and children

a)

0 index based mapping:

Parent : i , left child : 2i + 1; right child : 2i + 2

b)

1 index based mapping :

Parent : i , left child : 2i ; right child : 2i + 1

index	key
0	
1	50
2	30
3	70
4	20
5	40
6	60
7	80

Points:

- Compact representation if the tree is a complete tree
- If not complete, we should know which slots are not filled
- Might require a dummy value

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

- Each node has element, left and right
- Left and right are indices

Points:

- Require a memory manager to allocate and deallocate a free slot
- Otherwise insertion limited by the size of the array and not on available slots free on deletion

index	left	key	right
0	1	50	2
1	3	30	4
2	5	70	6
3	-1	20	-1
4	-1	40	-1
5	-1	60	-1
6	-1	80	-1

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

- Structure

```
#define MAXSIZE 200
struct node
{
        int key_;
        int left_;
        int right_;
};
typedef struct node node_t;
```

```
struct tree
{
        int root_;
        int count_;
        node_t elem_[MAXSIZE];
};
typedef struct tree tree_t;
```

- Count is used to indicate the next slot to insert as there is memory manager

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure
Interface: no change

```
void init(tree_t *ptr_tree);
void deinit(tree_t *ptr_tree);
void disp(tree_t *ptr_tree);
void insert(tree_t *ptr_tree, int key);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Initialize :

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = -1; // index of the first element; invalid to start
        ptr_tree->count_ = 0; // # of elem or slot to insert
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

insert :

```
void insert(tree_t *ptr_tree, int key)
{
// no space; exit
if(ptr_tree->count_  == MAXSIZE)
{
        printf("tree full\n");
        exit(1);
}
// count : empty slot at which key, left and right
// can be populated
// increment count to indicate the number of elem
// new node is always a leaf
int temp = ptr_tree->count_++;
ptr_tree->elem_[temp].key_ = key;
ptr_tree->elem_[temp].left_ =
        ptr_tree->elem_[temp].right_ = -1;
// call recursive to make the right connections
ptr_tree->root_ = insert_recursive(ptr_tree->elem_,
        ptr_tree->root_, temp);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Recursive insert :

```
static int insert_recursive(node_t elem[], int root, int temp)
{
        if(root == -1)
        {
                root = temp;
        }
        else if(elem[root].key_ > elem[temp].key_)
        {
                elem[root].left_ = insert_recursive(elem, elem[root].left_, temp);
        }
        else
        {
                elem[root].right_ = insert_recursive(elem, elem[root].right_, temp);
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Display:

```
void disp(tree_t *ptr_tree)
{
    printf("Tree : ");
    disp_recursive(ptr_tree->elem_, ptr_tree->root_);
    printf("\n");
}
```

```
static void disp_recursive(node_t elem[], int temp)
{
    if(temp != -1)
    {
            disp_recursive(elem, elem[temp].left_);
            printf("%d ", elem[temp].key_);
            disp_recursive(elem, elem[temp].right_);
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

```
# of nodes
int number_of_nodes(tree_t *ptr_tree)
{
  return   number_of_nodes_recursive(
ptr_tree->elem_,
              ptr_tree->root_);
}
```

```
int number_of_nodes_recursive(node_t elem[], int
temp)
{
   if(temp == -1)
   {
        return 0;
   }
   else
   {
        return 1 +
number_of_nodes_recursive(elem,
elem[temp].left_)       +
number_of_nodes_recursive(elem,
elem[temp].right_);
                }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

```
# of leaves
int number_of_leaves(tree_t *ptr_tree)
{
   return number_of_leaves_recursive
                (ptr_tree->elem_,
                ptr_tree->root_);
}
int number_of_leaves_recursive(node_t elem[], int temp)
{
                if(temp == -1)
                {
                                return 0;
                }
else if(elem[temp].left_ == -1 &&       elem[temp].right_ == -1)
{
                return 1;
}
else
{
    return number_of_leaves_recursive(
        elem, elem[temp].left_) +
number_of_leaves_recursive(
                elem, elem[temp].right_);
}
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Other functions:

- Traversals :
  - Inorder
  - Preorder
  - Postorder
- Compare two trees
- Copy tree

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)